University of Bremen
Department of Computer Science
Prof. Dr. h.c. Michael Beetz, PhD

AICOR Institute for Artificial Intelligence
WiSe 2026/2027
Worksheet 3

Exercises for the Lecture Advanced Methods of AI

Asking, Defining, Constraining, Deriving, Generating

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Last week you *represented* the pizza world in first-order logic and in Python. Tonight a service robot in a pizzeria *uses* that knowledge. Everything it does starts with one kind of expression, a **description**, written in the Entity Query Language (EQL). The five acts of the evening are five things the robot does with one:

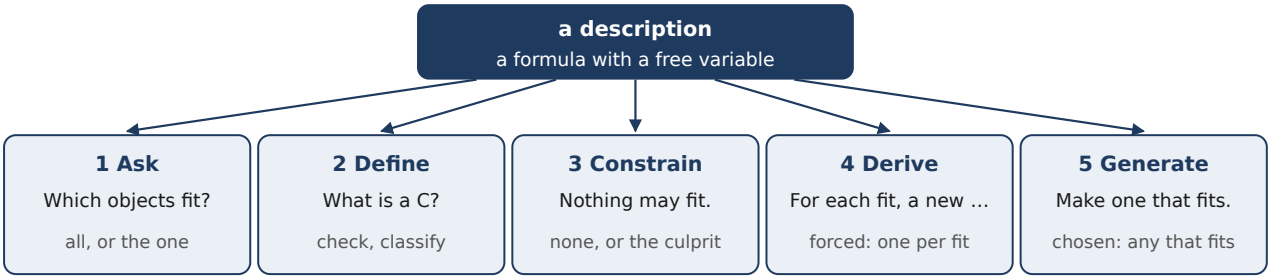


Figure 1. One description, five uses. The expression stays the same; what changes is what is done with it. Asking and constraining *read* what the robot knows; defining, deriving and generating *write* to it.

How to work

The evening happens in the notebook `03-pizza_queries.ipynb` (link on Stud.IP). Open it in pairs, run its setup cell, and work through it from top to bottom.

Three times, the tutor calls a **pen break**. Put the laptop aside and turn to this sheet: write the logic by hand for about six minutes. Then the tutor writes an answer on the board and takes your questions, and you go back to the notebook.

PEN BREAK 1 · AFTER ACT 1

Asking

About 6 minutes of writing, then the board.

(a) Not, and where it stands

Three formulas about a pizza x:

```
A:  ∃t (hasTopping(x, t) ∧ ¬FishTopping(t))
B:  ¬∃t (hasTopping(x, t) ∧ FishTopping(t))
C:  ∀t (hasTopping(x, t) → ¬FishTopping(t))
```

Mark which of them each pizza satisfies. Which two formulas agree on every pizza, and why? Which one did the `not_` path of Act 1 ask?

Pizza	A	B	C
napoletana_1: anchovies, capers, mozzarella, olives, tomato			
veggie_1: tomato, olives, mushroom			
plain_1: no toppings at all			

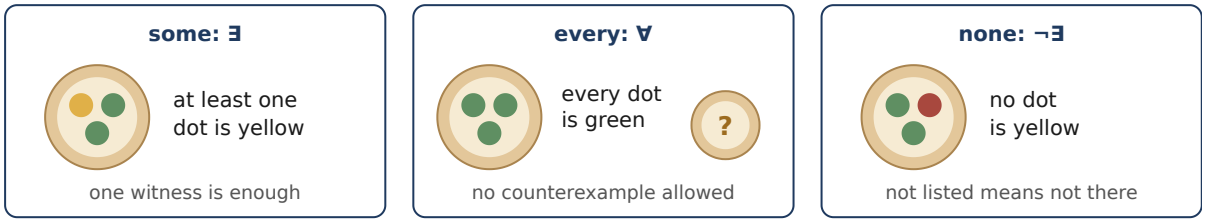


Figure 2. The three quantifier patterns, as the robot checks them on one pizza. The small pizza has no toppings at all: is every dot on it green?

(b) What the robot knows, and what follows

Q3, `none_is(CheeseTopping)`, returned veggie_1, and nobody ever stated "veggie_1 has no cheese". Give last week's OWL reasoner the same facts: hasTopping(veggie_1, t_1), TomatoTopping(t_1), and so on for the olives and the mushroom. Does it conclude that veggie_1 has no cheese? If not, what formula would you have to add, and why does the robot not need it?

PEN BREAK 2 · AFTER ACT 3

Defining and constraining

About 6 minutes of writing, then the board.

(a) Concluding from a name

The menu's definition from the notebook:

$$\forall x\ (\text{VegetarianPizza}(x) \leftrightarrow \text{Pizza}(x) \wedge \neg\exists t\ (\text{hasTopping}(x, t) \wedge (\text{MeatTopping}(t) \vee \text{FishTopping}(t))))$$

A pizza from the other branch arrives with a green V; nobody wrote down its toppings. What may the robot tell Chidi, who avoids fish? And Ben, who avoids milk? Which direction of ↔ do you use, and which direction did the robot use in Act 2 to put the green V on tonight's pizzas?

(b) A constraint is a query that must have no answer

The kitchen adds a second rule: *every order has at least one pizza*. Write it as a formula (with Order and hasPizza), and then its negation, with the ¬ pushed inwards. The robot checks the rule by asking the negation, its counterexamples. Which variables can an answer give values to? What, then, does an answer name, and how is that different from `dangerous`, which named an order *and* a pizza?

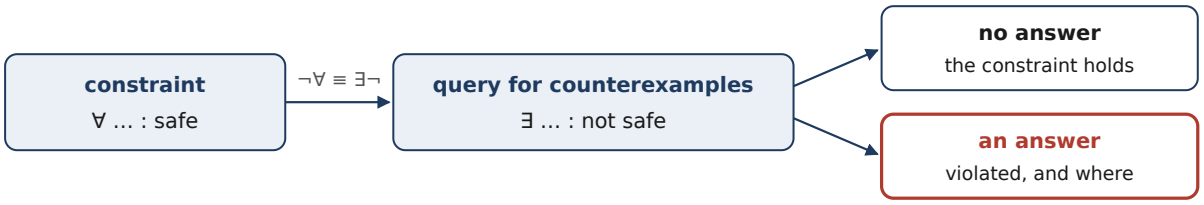


Figure 3. A universal statement is violated exactly when a counterexample exists.

PEN BREAK 3 · AFTER ACT 5

Deriving, and the whole evening

About 4 minutes for (a); then (b) together with the tutor.

(a) How many warnings?

The milk rule of Act 4, and a colleague's version of it:

```
R1:  ∀x (Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t))
         → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w) = milk))
R2:  ∀x ∀t (Pizza(x) ∧ hasTopping(x, t) ∧ CheeseTopping(t)
         → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w) = milk))
```

Are R1 and R2 equivalent formulas? The robot runs a rule by creating one new warning *w* for every answer of the rule's body, the part left of →. How many warnings does each rule write for a pizza with mozzarella *and* parmesan? Which part of the formula decides it?

(b) One allergy, five uses

Ben avoids milk. Write his allergy as each of the five uses of a description: a formula, or one sentence. The rule is done for you.

Use	Ben's milk allergy as …
1 Ask	
2 Define	
3 Constrain	
4 Derive	R1 of (a): a milk warning on every pizza with a cheese topping
5 Generate	

Take home

One expression, a description, carried the whole evening. Its meaning is a formula with free variables; what you *do* with it decides whether it is a question, a definition, a constraint, the body of a rule or a specification, and which backend answers it decides whether it is checked, computed or constructed.

To think about at home: a robot tidies a kitchen. Give one description for each of the five uses, and the backend that should answer it.